

Rapport d'audit — Guardian Ledger

Audit de sécurité, d'architecture et de performance

Dépôt : github.com/Ant0ine05/guardian_ledger · Branche : main (commit 50c8cf8) · Date : 11 juin 2026

Stack analysée : Vue 3 + Vite (frontend) · Express 5 + Prisma/SQLite (backend) · Nginx + Certbot + Docker Compose · CI/CD GitHub Actions → VPS.

L'architecture est globalement saine et bien découpée (routes / services / middleware, tests présents, CI fonctionnelle), mais l'audit révèle **4 failles critiques, dont une fuite de données réelle déjà publiée sur GitHub**, 7 problèmes importants et 10 points mineurs.

NIVEAU CRITIQUE

C1 users.db committée et poussée sur GitHub public — fuite de données réelle

`src/backend/data/users.db` est trackée par git et présente sur le dépôt public. L'inspection du blob commité confirme qu'il contient **des emails réels, des hashes bcrypt (\$2b\$12\$...) et des tokens OAuth Bungie (access + refresh)**. Le `.gitignore` racine liste bien le fichier, mais gitignore n'a aucun effet sur un fichier déjà tracké.

Commandes à exécuter immédiatement :

```
# 1. Détracker le fichier
git rm --cached src/backend/data/users.db
git commit -m "security: remove users.db from version control"

# 2. Purger TOUT l'historique (le fichier reste dans les anciens commits sinon)
pip install git-filter-repo
git filter-repo --invert-paths --path src/backend/data/users.db --force
git remote add origin https://github.com/Ant0ine05/guardian_ledger.git
git push origin --force --all

# 3. Révoquer les secrets exposés :
#   - Régénérer CLIENT_SECRET et BUNGIE_API_KEY sur bungie.net/en/Application
#   - Régénérer JWT_SECRET sur le VPS
#   - Vider les colonnes bungieAccessToken / bungieRefreshToken en base prod
#   - Forcer un reset de mot de passe pour les comptes concernés
```

C2 Secret JWT par défaut hardcodé — forge de tokens possible

`routes/routerAuth.js:7` et `middleware/requireAuth.js:2` :

```
const JWT_SECRET = process.env.JWT_SECRET || 'guardian-ledger-dev-secret-changeme';
```

Ce fallback est public sur GitHub. Si `JWT_SECRET` manque dans l'environnement de production (faute de frappe dans le `.env`, oubli), **n'importe qui peut forger un appToken valide** et accéder aux comptes Bungie liés. Un secret manquant doit faire crasher le serveur, pas dégrader silencieusement :

```
// config commune (ex : src/backend/config.js)
const JWT_SECRET = process.env.JWT_SECRET;
if (!JWT_SECRET || JWT_SECRET.length < 32) {
  throw new Error('JWT_SECRET manquant ou trop court - démarrage refusé.');
```

C3 Callback OAuth : state facultatif + token de session dans l'URL

routes/routerAuth.js:99-111 — deux problèmes combinés :

- **state est optionnel** : s'il est absent, le code continue (`userId = null`), retombe sur `findUnique({ where: { bungieMembershipId } })` (ligne 182) et **émet un JWT valable 7 jours**. Un attaquant peut forger l'URL `/api/auth/callback?code=<son propre code Bungie>` sans state — c'est aussi un vecteur de *login CSRF*. Même chose si `decoded.type !== 'link_bungie'` : aucun rejet.
- **Le JWT final part dans la query string** (ligne 198) : `?appToken=...` finit dans l'historique navigateur, les logs nginx et les en-têtes Referer.

```
router.get('/callback', async (req, res) => {
  const { code, state } = req.query;
  if (!code) return res.status(400).send('Code OAuth manquant.');
```

Pour le token dans l'URL, a minima passer par un **fragment** (jamais envoyé au serveur ni dans le Referer) :

`res.redirect(`${FRONTEND_URL}/dashboard#appToken=${appToken}`)` et côté `Dashboard.vue:289` lire `new`

`URLSearchParams(location.hash.slice(1)).get('appToken')` . Solution cible : cookie `httpOnly; Secure; SameSite=Lax` .

C4 Build Docker cassé / dangereux : pas de .dockerignore + Node 18 incompatible

- Aucun `.dockerignore` n'existe : `COPY . .` embarque dans l'image **node_modules Windows** (le binaire natif better-sqlite3 compilé pour win32 écrase celui installé pour Alpine → crash au démarrage), **manifest.db (305 Mo)** et **users.db**.
- `src/frontend/Dockerfile:2` utilise `node:18-alpine` alors que le package.json frontend exige `^20.19.0 || >=22.12.0` (Vite 7 refuse Node 18). Prisma 7 côté backend exige aussi Node ≥ 20.

```
# src/backend/Dockerfile et src/frontend/Dockerfile – ligne 1
FROM node:22-alpine

# src/backend/.dockerignore (à créer)
node_modules
data/
.env
*.db

# src/frontend/.dockerignore (à créer)
node_modules
dist
```

Dans le Dockerfile backend, remplacer `RUN npm install` par `RUN npm ci` (build reproductible, comme en CI).

● NIVEAU IMPORTANT

I1 CORS ouvert à tous + aucun rate-limiting + pas de headers de sécurité

`server.js:6` : `app.use(cors())` autorise toutes les origines, et `/api/auth/login` / `register` n'ont aucune protection brute-force (bcrypt cost 12 rend chaque tentative coûteuse pour le serveur aussi → vecteur DoS).

```
const rateLimit = require('express-rate-limit'); // npm i express-rate-limit helmet
const helmet = require('helmet');

app.use(helmet());
app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));
app.set('trust proxy', 1); // derrière nginx, sinon req.ip = IP du proxy

const authLimiter = rateLimit({ windowMs: 15 * 60 * 1000, limit: 20 });
app.use('/api/auth/login', authLimiter);
app.use('/api/auth/register', authLimiter);
```

I2 Rafraîchissement du site cassé en prod (404 sur /dashboard)

Le conteneur frontend sert la SPA avec la config nginx **par défaut** (`src/frontend/Dockerfile:12-15`) : pas de fallback history-mode. Un F5 sur `/dashboard` ou `/vault` → 404. D'autant plus grave que le flux OAuth **atterrit précisément sur /dashboard?appToken=...** : la liaison Bungie est donc cassée en production.

```
# src/frontend/nginx.conf (à créer)
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    gzip on;
    gzip_types text/css application/javascript application/json image/svg+xml;

    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
    location / {
        try_files $uri $uri/ /index.html;
    }
}

# src/frontend/Dockerfile – ajouter avant CMD
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

I3 Goulot d'étranglement majeur : manifestService.js

`services/manifestService.js:7-13` cumule trois problèmes pour la route la plus chaude du site (`/api/me/destiny` fait des **centaines** d'appels `getDefinition` par requête — items + plugs + plugsets) :

- `verbose: console.log` → **chaque requête SQL est loggée en prod** (I/O console synchrone, énorme ralentissement) ;
- `db.prepare(...)` est recompilé **à chaque appel** au lieu d'être mis en cache ;
- `JSON.parse` répété sur les mêmes définitions, sans cache ;
- bonus : nom de table interpolé dans le SQL — non exploitable aujourd'hui (appelants hardcodés), mais une whitelist coûte 3 lignes.

```

const Database = require('better-sqlite3');
const path = require('path');

const dbPath = path.resolve(__dirname, '../data/manifest.db');
const db = new Database(dbPath, { readonly: true, fileMustExist: true });

const ALLOWED_TABLES = new Set([
  'DestinyInventoryItemDefinition', 'DestinyInventoryBucketDefinition',
  'DestinyStatDefinition', 'DestinyStatGroupDefinition', 'DestinyPlugSetDefinition',
]);

const stmtCache = new Map(); // table → prepared statement
const defCache = new Map(); // "table:hash" → définition parsée

const getDefinition = (tableName, hash) => {
  if (!ALLOWED_TABLES.has(tableName)) return null;
  const key = `${tableName}:${hash}`;
  if (defCache.has(key)) return defCache.get(key);
  try {
    let stmt = stmtCache.get(tableName);
    if (!stmt) {
      stmt = db.prepare(`SELECT json FROM ${tableName} WHERE id = ?`);
      stmtCache.set(tableName, stmt);
    }
    const row = stmt.get(hash >> 0);
    const def = row ? JSON.parse(row.json) : null;
    if (defCache.size > 20000) defCache.clear(); // garde-fou mémoire
    defCache.set(key, def);
    return def;
  } catch (error) {
    console.error('Erreur SQL:', error);
    return null;
  }
};

module.exports = { getDefinition };

```

14 Course critique sur le refresh token Bungie

[routes/routerMe.js:65-102](#) — Bungie **fait tourner le refresh token à chaque usage**. Or la file de transferts côté Vault + l'ouverture d'une modale de détail peuvent déclencher plusieurs requêtes simultanées : deux `ensureFreshToken` concurrents consomment le même refresh token → le second échoue et peut **invalider la session Bungie de l'utilisateur** (le symptôme correspond au commit « requete en parralle a verifier in game »). Mutex par utilisateur :

```

const refreshLocks = new Map(); // userId → Promise en cours

async function ensureFreshToken(user) {
  const expiresAt = user.bungieTokenExpiresAt ? new Date(user.bungieTokenExpiresAt) : null;
  if (expiresAt && Date.now() < expiresAt.getTime() - 5 * 60 * 1000)
    return user.bungieAccessToken;

  if (refreshLocks.has(user.id)) return refreshLocks.get(user.id);

  const job = (async () => {
    try {
      // ... corps actuel du refresh, inchangé ...
      return newAccessToken;
    } finally {
      refreshLocks.delete(user.id);
    }
  })();
  refreshLocks.set(user.id, job);
  return job;
}

```

15 destinyMemberships[0] ignore le cross-save

`routes/routerMe.js:126` (idem lignes 271, 427, 482) : un joueur cross-save a plusieurs memberships et `[0]` n'est pas garanti être le bon → coffre vide ou mauvaise plateforme. À corriger aux 4 endroits (ou mieux : factoriser en un helper `getDestinyMembership(user, headers)`) :

```
const r = membRes.data.Response;
const primary = r.destinyMemberships.find(
  m => m.membershipId === r.primaryMembershipId
) || r.destinyMemberships.find(m => m.crossSaveOverride === m.membershipType)
  || r.destinyMemberships[0];
const { membershipId, membershipType } = primary;
```

16 Mauvaise table de manifest dans routerData.js — bug logique + route sans auth

`routes/routerData.js:6-8` : la route s'appelle `/item/:hash`, l'exemple en commentaire (2264350288) est un hash d'**item**, mais la requête interroge `DestinyStatDefinition`. C'est exactement le genre de coquille recherchée : la route renvoie 404 (ou pire, une stat homonyme) pour tout vrai item. De plus `item.displayProperties.name` crashe si `displayProperties` est absent, et la route est publique.

```
router.get('/item/:hash', (req, res) => {
  const item = getDefinition('DestinyInventoryItemDefinition', req.params.hash);
  if (!item) return res.status(404).json({ error: 'Item introuvable' });
  res.json({
    name: item.displayProperties?.name ?? 'Inconnu',
    icon: item.displayProperties?.icon
      ? `https://www.bungie.net${item.displayProperties.icon}` : '',
    type: item.itemTypeDisplayName ?? '',
  });
});
```

17 Aucune stratégie de mise à jour du manifest

`manifest.db` (305 Mo) n'est ni commité ni téléchargé au déploiement : il doit être copié à la main dans le volume, et **devient périmé à chaque patch Destiny** (les définitions changent toutes les ~6 semaines ; les nouveaux items renverront `null` → items invisibles dans le coffre). Ajouter un script de bootstrap qui interroge `GET /Destiny2/Manifest/`, compare la version stockée, télécharge le `mobileWorldContentPaths.fr` et remplace le fichier — exécuté au démarrage du conteneur et/ou via cron.

NIVEAU MINEUR

#	Problème	Correctif
M1	Code mort : <code>services/bungieService.js</code> jamais importé ; <code>component/Modal.vue</code> fait 0 octet ; <code>views/HomeView.vue</code> n'est dans aucune route ; <code>showToast</code> de <code>Dashboard.vue:307</code> jamais appelé ; styles header/nav de <code>App.vue</code> orphelins ; <code>routerData</code> jamais appelé par le front	<code>git rm</code> les fichiers, supprimer le code inutilisé
M2	Ligne de debug oubliée en prod : <code>ItemDetailModal.vue:20</code> affiche <code>itemHash: ...</code> en orange monospace dans la modale	Supprimer la ligne
M3	Coquille dans <code>src/backend/.gitignore</code> : <code>data/user.db</code> (singulier) au lieu de <code>users.db</code> — c'est en partie pour ça que le fichier a fini commité	Corriger en <code>data/*.db</code>
M4	Duplication massive : <code>parseJwt</code> / <code>applyUser</code> / <code>showToast</code> / <code>fetchDestinyData</code> copiés-collés entre <code>Dashboard.vue</code> et <code>Vault.vue</code> ; <code>CLASS_MAP</code> redéclaré inline dans <code>routerMe.js:51</code>	Extraire un composable <code>useAuth()</code> + <code>useDestinyData()</code> ; réutiliser <code>CLASS_MAP</code>
M5	<code>membershipCache</code> (<code>routerMe.js:28</code>) : Map non bornée, jamais invalidée (un re-link Bungie vers un autre compte garde l'ancien membership en cache)	Invalider l'entrée dans le callback OAuth ; borner la taille
M6	nginx public : pas de HSTS, pas de gzip, pas de http2	Ajouter <code>add_header Strict-Transport-Security "max-age=31536000" always; , gzip on; , listen 443 ssl http2;</code> dans <code>nginx/nginx.conf</code>
M7	« Season of the Cipher · S26 » hardcodé à 3 endroits	Constante unique ou donnée API
M8	<code>src/backend/package.json:4</code> : <code>"main": "index.js"</code> mais le fichier est <code>server.js</code>	<code>"main": "server.js"</code>
M9	Surbrillance drag collante : les <code>equip-slot</code> ont <code>@dragover</code> mais pas de <code>@dragleave</code> (<code>Vault.vue:106</code>)	Ajouter <code>@dragleave="dragOverTarget = null"</code>
M10	Pas de middleware d'erreur global Express ni de route <code>GET /health</code> (utile pour le healthcheck Docker/CI)	<code>app.get('/health', (_, res) => res.json({ ok: true })))</code> + handler d'erreur final

SYNTHÈSE

Priorité	Nombre	Thème dominant
● Critique	4	Fuite de données déjà publique (C1) — à traiter aujourd'hui : purge git + rotation des secrets
● Important	7	Durcissement auth/CORS, prod cassée (SPA 404, Node 18), perfs manifest
● Mineur	10	Code mort, duplication, hygiène config

L'architecture (séparation routes/services/middleware, file de transferts optimiste côté Vault, tests + CI) est solide pour un projet de cette taille. Les deux fichiers à refactorer en priorité après les correctifs de sécurité : `routerMe.js` (factoriser le trio `ensureFreshToken` / `membership` / `headers` répété 4x) et le duo `Dashboard.vue` / `Vault.vue` (composables partagés).